Management System ISO 9001:2015
Level III Recognition for Mastery in Quality Management 2022
THE Impact Rankings 2023 +1001
Dark Green School Certified
World University Rankings Ranked 684
The WORLD UNIVERSITY RANKINGS for INNOVATION

RODRIGUEZ, MILES
BSIT 2H-G1

JAVA STACK SAMPLE PROBLEMS

General Learning Objectives

After completing these activities, students should be able to:

1. Create a Stack using java.util.Stack.
2. Add items to a stack using push(). 3. Remove the top item using pop().
4. View the top item using peek().
5. Check the number of items using size().
6. Check whether a stack is empty using isEmpty().
7. Search for an item using contains().
8. Explain how the stack changes after each operation and apply the Last-In, First-Out (LIFO) principle.

GENERAL INSTRUCTIONS

For each problem, follow these rules:

1. **Import Stack:** Your program must use: import java.util.Stack;
2. **Use your name in the class name:** The class name must follow this format: StackProb_FirstnameLastname
    - *For example, if the student's name is Juan Dela Cruz:* StackProb_JuanDelaCruz.java
3. **File name:** The Java file must have exactly the same name as the public class name.
4. **Use comments:** Students should add inline comments to identify important stack operations.
5. **Submit:** For each problem, students should submit:
    - .java source code
    - Screenshot of the program output
    - Answers to the reflective questions

SAMPLE PROBLEM 1 — Student Name Stack

Create a Java program that uses a Stack<String> to store student names representing a processing pile. Remember that the last name added is placed at the top.

The program should:

1. Create an empty Stack<String> named students.
2. Push these five names in order: **Ana, Ben, Carlo, Diana, Eric**.
3. Display the original stack layout.
4. Display the top name using peek().
5. Remove the top name using pop() and display the name that was removed.
6. Display the stack after removal.
7. Display the total number of names remaining using size().

Expected Output:

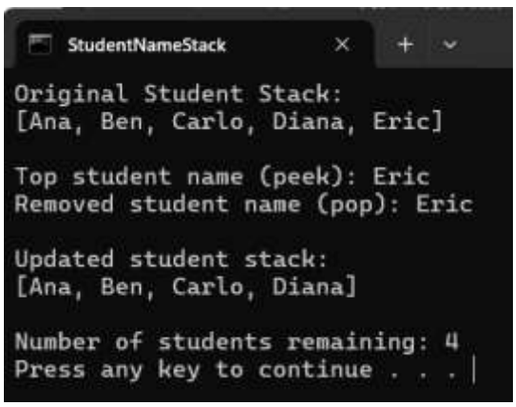
```
Original Student Stack:
[Ana, Ben, Carlo, Diana, Eric]

Top student name (peek): Eric
Removed student name (pop): Eric

Updated Student Stack:
[Ana, Ben, Carlo, Diana]

Number of students remaining: 4
```

Screenshot Output:



SAMPLE PROBLEM 2 — Browser Back History

Create a Java program that uses a Stack<String> to represent a simple browser history. Each page visited is pushed onto the stack, making the most recently visited page sit at the top.

The program should:

1. Start with these pages, pushing them in order: **Home, Student Portal, Course Materials, Java Lessons**.
2. Display the original browser history stack layout.
3. Display the current page using peek().
4. Push a new page named **Quiz Page** onto the stack.
5. Display the stack layout after visiting the Quiz Page.
6. Use contains() to check whether "**Student Portal**" is available in the stack history and display the result.
7. Simulate pressing the browser's "Back" button by using pop(). Display the page that was removed.
8. Display the current page using peek() after shifting back.
9. Display the final stack history and the number of pages remaining using size().

Expected Output:

Management System ISO 9001:2015
Level III Recognition for Mastery in Quality Management 2022
THE Impact Rankings 2023 +1001
Dark Green School Certified
World University Rankings Ranked 684
The WORLD UNIVERSITY RANKINGS for INNOVATION

```
Original Product List:
[Home, Student Portal, Course Materials, Java Lessons]

Current page: Java Lessons

After visiting Quiz Page:
[Home, Student Portal, Course Materials, Java Lessons, Quiz Page]

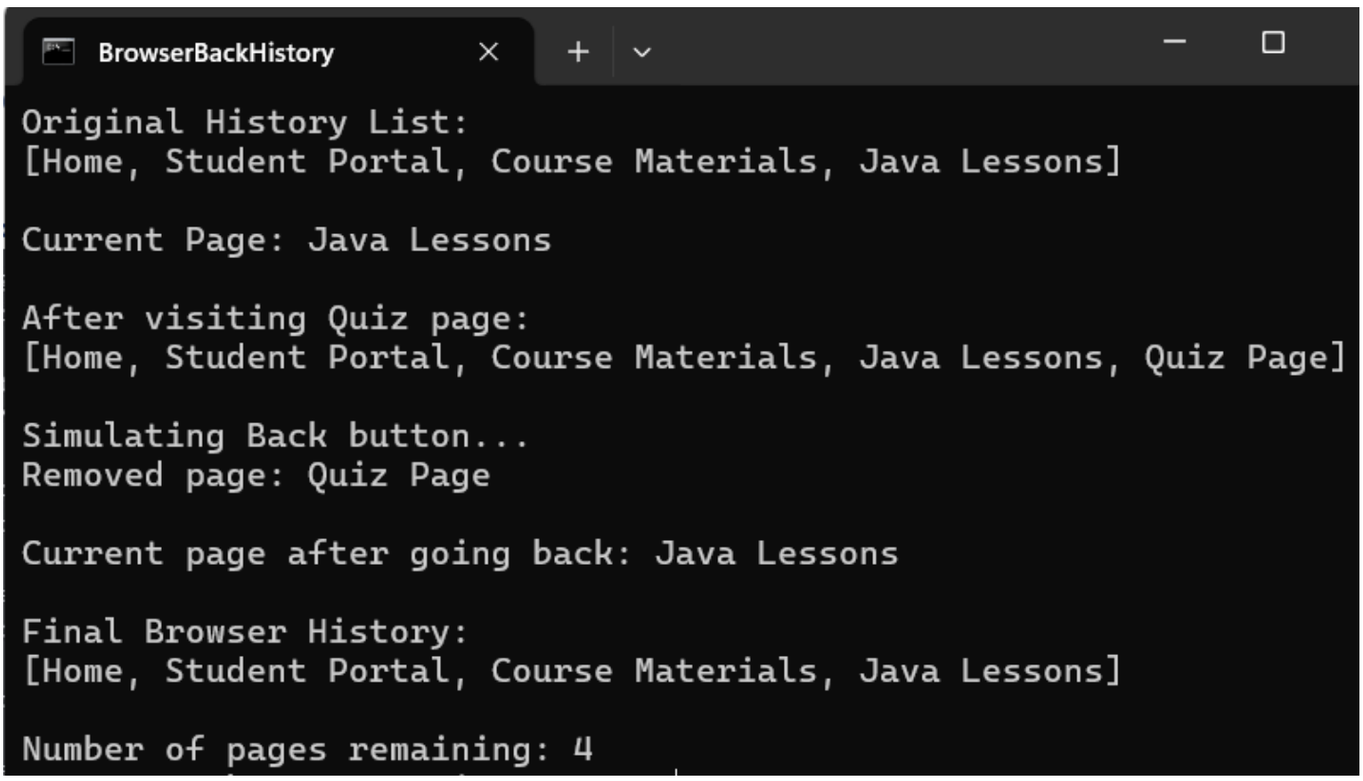
Is Student Portal in history? true

Simulating Back button...
Removed page: Quiz Page

Current page after going back: Java Lessons

Final Browser History:
[Home, Student Portal, Course Materials, Java Lessons]
Number of pages remaining: 4
```

Screenshot Output:



SAMPLE PROBLEM 3 — Undo Recent Actions

Create a Java program that uses a Stack<String> to represent actions performed in a simple text editor. Each new action is pushed onto the stack, and the most recent action must be undone first.

The program should:

1. Push these actions in order: **Typed title, Added paragraph, Inserted image, Changed font size, Changed text color**.
2. Display all actions currently in the text editor stack.
3. Display the most recent action using peek().
4. Check whether the stack contains the action "**Added paragraph**" using contains(), displaying the result.
5. Undo the most recent action using pop() and display the action undone.
6. Undo one more action using pop() and display the action undone.
7. Display the remaining actions left in the workspace.
8. Display the number of actions remaining using size().
9. Continue removing items one at a time until the stack is entirely empty. Before each pop(), use isEmpty() to avoid throwing runtime errors.
10. Display the final stack layout and the result of isEmpty() at the end.

Expected Output:

```
Original Student List:
[Typed title, Added paragraph, Inserted image, Changed font size, Changed text c

Most recent action: Changed text color
Is 'Added paragraph' found? true

Undoing most recent action: Changed text color
Undoing next recent action: Changed font size

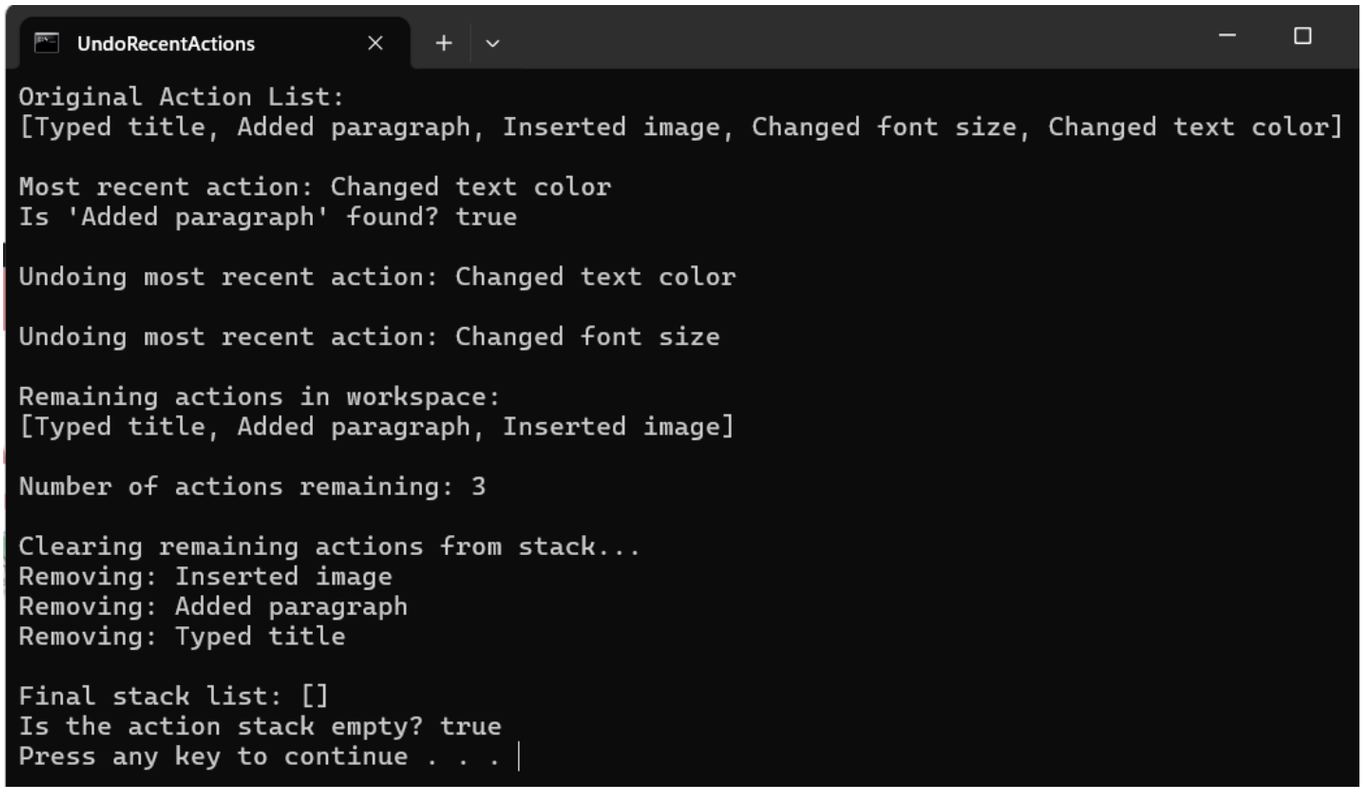
Remaining actions in workspace:
[Typed title, Added paragraph, Inserted image]

Number of actions remaining: 3

Clearing remaining actions from stack...
Removing: Inserted image
Removing: Added paragraph
Removing: Typed title

Final Stack List: []
Is the action stack empty? true
```

Screenshot Output:



REFLECTIVE QUESTION

After completing the three problems, students should answer the following questions in their own words:

1. What is the purpose of: import java.util.Stack;?

   We need to import java.util.Stack to let us use methods essential for stack operations. Without importing it, we cannot use push() to add an element, pop() to remove an element, and more in a stack.

2. What does push() do to a stack structural arrangement?

   Push() puts new elements on top of a stack. Therefore, changing the arrangement by placing the former top element beneath the newly placed element.

3. What is the difference between pop() and peek()?

   Pop() removes the top element in a stack, while peek() allows us to look on what's on the top of the stack without removing it.

4. What does size() tell us about an active stack collection?

Size() allows us to see how many elements are pushed and is still existing in a stack.

5. What does isEmpty() return when a stack contains no elements?

   isEmpty() would return true when no elements are existing within a stack

6. What does contains() check inside your program?

   Contains() are used to find if a specific value is existing within the stack

7. What does LIFO mean? Explain it briefly using one of the lab problems.

   LIFO means “Last in, first out”. For instance of student names, when the last student, which is Eric, were added on the top of the stack, unless nothing else is pushed after his name, when we call pop() or peek(), Eric’s name would be first to be called.

8. Why should a program check isEmpty() before executing a pop() method call?

   A program should check if a stack is empty using isEmpty() before pop() to make sure that there is indeed something to be removed in the stack. Using pop() or removing a supposed element in an empty stack would only cause an error called EmptyStackException.

9. Give one real-life example of a stack outside of computing and explain how it uses LIFO.

   I would say the game Tower of Hanoi is a good example of stack, as its top pieces (the one placed last) are the only pieces you can move in the game.

10. Which stack method or operation did you find easiest to understand? Explain why.

    I find the push() stack method the easiest to understand as it is just adds elements on a stack in a straightforward manner.

JAVA STACK ACTIVITY RUBRIC — 100 POINTS

Criteria	Excellent (21–25 pts)	Good (16–20 pts)	Developing (11–15 pts)	Beginning (0–10 pts)	Score
Program Requirements	All required operations for the three problems are implemented correctly.	Most requirements are implemented with minor deviations.	Several core lab requirements are missing.	Major requirements are missing or unformed.	**/ 25**
Correct Use of Stack Methods	Correctly applies methods like push(), pop(), peek(), contains(), size(), and isEmpty().	Minor syntax errors in stack method usage.	Several method applications throw errors.	Methods are largely incorrect, missing, or bypassed.	**/ 25**
Program Output	Output is complete and matches the required console operations.	Minor text or format output issues.	Some console output is missing or incorrect.	Output data is largely incorrect or unrunnable.	**/ 20**
Code Organization	Code is properly structured, readable, and includes useful comments.	Generally organized with minor commenting gaps.	Code structure is cluttered or difficult to trace.	Difficult to follow; comments are completely missing.	**/ 10**
Naming Convention	Correct class and file naming convention using StackProb_FirstnameLastname .	Minor typo in naming convention formatting.	Several naming issues on class or files.	Does not follow the required naming structure at all.	**/ 5**
Reflection	Answers are clear, complete, and demonstrate structural understanding.	Answers show adequate conceptual understanding.	Reflection answers are brief or incomplete.	Answers show little understanding or are plagiarized.	**/ 10**
Program Demonstration	Complete screenshots of working code and matching console output are provided.	One minor missing capture item.	Several code verification screenshots are missing.	No proper evidence or screenshots provided.	**/ 5**

TOTAL SCORE: _____ / 100 Points